

The Alnception dataset



**Funded by
the European Union**

This work was funded by the European Union as part of the European Defence Fund (EDF) project Alnception (GA No. 101103385). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union (EU). The EU cannot be held responsible for them.

The Alnception project commenced in 2022 with completion in 2025, with the following partners: Space Hellas (Greece), FFI (Norway), Thales SIX GTS (France), Naval Group (France), Ubitech (Greece), Eight Bells (Cyprus), Infini (Greece), FOI (Sweden), TNO (the Netherlands), KTH (Sweden), NCSR “Demokritos” (Greece), AIT (Austria), HMOD (Greece), BDI (Bulgaria), CEA (France), CY4Gate (Italy), mnemonic (Norway), Thales (Greece). Subcontractors: University of Oslo (Norway), Logstail (Greece), WebIQ (the Netherlands) and CapGemini (the Netherlands).

Alnception’s main aim was to develop novel AI-based tools and techniques for detection and response for military IT/OT systems: detecting adversarial behaviour from logs and network traffic; understanding, contextualising and explaining the detected threats; generating risk and impact aware response action; and automating the execution and evaluation of the response action on the underlying infrastructure. Selected tools were combined into a proof-of-concept end-to-end detection and response prototype, evaluated in operational scenarios with end users. More details about the Alnception project can be found here:

<https://www.ainception.eu/>

To ensure military realism and relevance, an underlying military context with a military scenario and several military storylines were developed. This underlying military context, consisting of a military scenario with a consistent timeline of several storylines, can be found in the following report [1]:

A. Thuv (editor) et al. Alnception scenario and storylines, FFI-report 25/039, Available from <https://www.ffi.no/en/publications-archive/ainception-scenario-and-storylines>, 2025

This document describes the publicly released AIInception dataset. The dataset contains relevant data from the simulation and use of three AIInception (military) storylines: SL100, SL300, and SL700.

The dataset comprises 15 simulations, with each ZIP file containing a single simulation. Each ZIP file is a variant of one of the storylines. The different variants may differ in both the benign and malicious behaviour and the underlying infrastructure. However, they all align with the underlying storyline.

This document provides a brief overview of how the storylines have been simulated to facilitate understanding of the dataset and details the file structure of the associated ZIP files.

Limitations

The dataset has been developed for the purpose of supporting development and evaluation within the AIInception project, and has not been adapted further for public release. In other words, it has been published “as-is” and thus comes with some limitations. One example is that alignment with storylines and addressing of the project objectives have been prioritised over a high-quality dataset and complex attacks. This means that certain aspects have been simplified compared with real attacks and systems. We have not been able to spend much effort on “cleaning” the dataset to make it easier to use. Despite these limitations, we believe it can be a valuable resource for research projects addressing cyber and military usage, in particular when also taking into account the underlying military context and storylines [1]. In fact, we believe this complete perspective from a geo-political scenario – through the operational and tactical levels, as detailed in the timeline and storylines [1] – all the way to the dataset at the technical level described here, is unique.

Simulation of the storylines

Here, we describe how SL100, SL300 and SL700 have been simulated to facilitate understanding of the dataset.

SL100

SL100 simulates a border monitoring operation that is conducted using advanced drones for continuous surveillance, all managed from a central control station equipped with cybersecurity tools to protect both the drone fleet and the data gathering pipelines. SL100 was generated from the network topology outlined in Figure 1. It is divided into two main subnets:

- **192.168.1.0/24:** This is the subnet where the Windows machines run.
- **192.168.2.0/24:** This subnet is dedicated to the Drone Simulator and the Security Operations Centre (SOC) components.

The simulation included the following key components:

- **Windows Server:** A Windows server used to manage the other windows machines of the topology and push updates.
- **PC2:** A Windows 10 machine used as the main UAV controller/operator. This is the machine that is compromised to gain access to the UAV.
- **PC1:** A Windows 10 secondary machine acting as another UAV controller.

- **Firewall:** This Opnsense firewall that mainly acts as a router between the two subnets.
- **Drone Simulator:** An Ubuntu machine where the drone simulator runs. This is the ultimate target of the simulated attack. For the drone simulation, the [ArduPilot](#) open-source software [2] was used.
- **Suricata, Kafka, Logstash, Opensearch:** These components are part of AIInception's SOC pipeline and are used to collect logs and data from the machines affected by the attack.

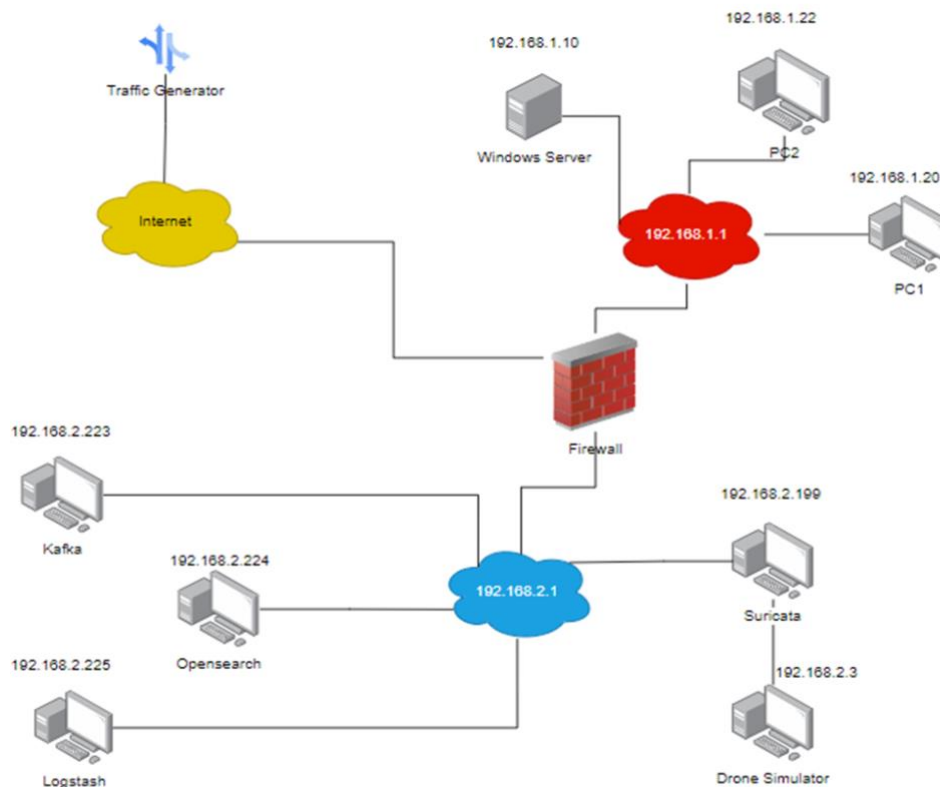


Figure 1. Network topology for SL100.

The simulated storyline covers the following:

- *Generation of data with only benign events.* In this case, the Unmanned Aerial Vehicle's (UAV's) mission was not intercepted and no malicious attempt against the infrastructure was performed.
- *Generation of data with malicious events.* The infrastructure was compromised, and the UAV was intercepted after completing half of its mission plan.

The above data are part of a single dataset with a time gap between the benign events and the beginning of the malicious ones. The aim was to include simulations of how a UAV operator would perform patrol missions with an unexpected attack against the drone during one of those missions. Therefore, two patrol missions over the designated area were completed, with the drone successfully completing both. Thereafter, the first steps of the attack were initiated, which included network scans and pivoting between machines. In the meantime, the drone operator initiated a third mission. Halfway through this mission, a connection was made to the UAV machine causing a termination of the UAV.

SL100's topology infrastructure was captured in a graph database using Neo4j. The accompanying file¹ contains a Neo4j export of the SL100 infrastructure graph. It represents the network topology used in the simulation, including all key devices, their IP addresses and the logical relationships between them. The data follow a STIX 2.1 schema and consist mainly of Infrastructure nodes (e.g., hosts and network devices) and IPv4Address nodes (individual IPs and CIDR ranges), linked by relationships that describe composition and connectivity.

Each Infrastructure node corresponds to a component of the SL100 environment and is characterized by properties such as name, infrastructure_types, and id. Examples include the firewall *FW-SL100* (infrastructure_types: firewall), the routers *ROUTER_1* and *ROUTER_2* (routers-switches), the Windows-based hosts *windows_server*, *pc1*, *pc2*, the drone-related host *drone_simulator*, and the SOC components *Suricata*, *Kafka*, *Logstash* and *OpenSearch* (all modelled as workstation infrastructure). Separate IPv4Address nodes capture both interface addresses (e.g. 192.168.1.10, 192.168.1.22, etc., and subnet ranges (192.168.1.0/24 and 192.168.2.0/24).

The relationships in the JSON file define how these objects fit together in the network. The *consists_of* relationships link infrastructure nodes to their assigned IP addresses and to the subnets to which they belong. The *communicates_with* relationships capture expected communication paths between devices, such as hosts talking to their respective routers, routers linked to their subnets, and SOC components communicating via *ROUTER_2*. Together, these nodes and relationships form a complete graph representation of the SL100 infrastructure described in the storyline.

SL300

Simulation procedure

The simulations of SL300 were divided into two distinct parts:

- *Generation of training data with only benign events.* This amounted to five days of simulation.
- *Generation of test data where malicious events were mixed with benign events.* This amounted to one day of simulation.

As simulation of the malicious events risked to fail, systems were shut down for a virtual machine snapshot between these parts. Thus, there is a time gap between the two parts in each dataset. As illustrated in table 1, four benign events were simulated in four different ways and attacks were simulated in ten different ways. The cyber infrastructures differ by the name of the domain, but are otherwise identical.

Table 1. Overview of the simulations SL300.

Cyber infrastructure	Benign events	Malicious events	Total duration
bnhq1.rotso (variant 1)	Variant A	Variant 1	6 days
bnhq2.rotso (variant 2)	Variant A	Variant 2	6 days

¹ See *SL100_infra.json* in *SL100.zip* (further detailed below).

bnhq3.rotso (variant 3)	Variant A	Variant 3	6 days
bnhq4.rotso (variant 4)	Variant B	Variant 4	6 days
bnhq5.rotso (variant 5)	Variant B	Variant 5	6 days
bnhq6.rotso (variant 6)	Variant B	Variant 6	6 days
bnhq7.rotso (variant 7)	Variant C	Variant 7	6 days
bnhq8.rotso (variant 8)	Variant C	Variant 8	6 days
bnhq9.rotso (variant 9)	Variant C	Variant 9	6 days
bnhq10.rotso (variant 10)	Variant D	Variant 10	6 days

Disk space caused problems with the NetFlow recordings of simulations 1 and 3 listed in table 1. As a result, no Suricata logs were included in those two simulations. Due to this limitation, we have not included variant 1 and variant 3 in the dataset. No other known issues occurred.

Cyber infrastructure

The cyber infrastructure in SL300 represents a temporary battalion headquarter (BnHQ) located at Rotso National Airport and its interaction with mechanized infantry. The BnHQ encompasses a small-scale deployed IT network that is largely built on mainstream Windows technology. The network incorporates a number of workstations, an Active Directory (AD) and an application server that hosts the military C2 application. External connectivity is limited to the minimum necessary. However, a permanent VPN-over-satellite uplink is set up. Three vehicles in the mechanised infantry companies are command vehicles. These regularly send updates on their position and direction to the military C2 system to facilitate blue force tracking. When the vehicles are at rest, they communicate images and videos to the BnHQ infrastructure over satellite links.⁵

The network topology is outlined in figure 1. The key components and the host names are as follows:

- The Windows clients hq1-hq20 in the segment (CLIENTS) where simulated users reside.
- The domain controller (dc) for all Windows machines in BNHQ.
- Email servers (mail and mailrelay) and their certificate servers (ca and rootca).
- The simulated C2 system (apps) along with its webapp (monitor)
- File server for reconnaissance (media), mostly pictures.
- The simulated vehicles (pv1c2, pv2c2, and pv3c2).
- The firewall (fw1) with open ports to the satellite and vehicles.

The figure also shows the SOC segment where systems used to collect logs are located. Logs are forwarded from all machines except the simulated vehicles.

In the simulation, some shortcuts have been taken. The military C2 application is a mock-up created for this storyline. It involves a message queue, a basic database, and a web-based user interface. The functionality of this mock-up is not as complex as a real C2. For instance, vehicles send locations based on a simulated route and upload random media files occasionally, but media files do not correspond to a real route or a predefined one.

Furthermore, all interaction with the vehicles is through the radio interface, including the media files (that are sent over satellite in the storyline).

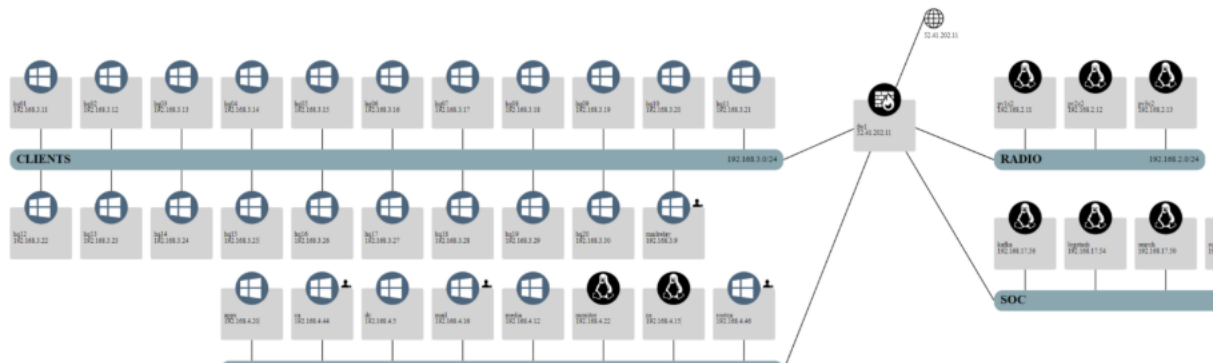


Figure 2. Network topology for the temporary headquarters (SL300; bnhq1.rotso).

The dataset contains facts files from the provisioning and configuration tool *Ansible* and a network scan from one of the networks (all are identical). An excerpt of the facts file is provided below.

```
{
  "ansible_facts": {
    "all_ipv4_addresses": [
      "192.168.17.50"
    ],
    "all_ipv6_addresses": [
      "fe80::208:74ff:fe25:155d"
    ],
    "ansible_local": {
      "os_type": ""
    },
    "apparmor": {
      "status": "enabled"
    },
    "architecture": "x86_64",
    "bios_date": "12/01/2006",
    "bios_vendor": "innotek GmbH",
    "bios_version": "VirtualBox",
    "board_asset_tag": "NA",
    "board_name": "VirtualBox",
    "board_serial": "0",
    "board_vendor": "Oracle Corporation",
    "board_version": "1.2",
    "chassis_asset_tag": "NA",
    "chassis_serial": "NA",
    ...
  }
}
```

Benign events in the cyber infrastructure

There are two types of benign activity in the simulation:

- Simulated messaging to and from the vehicles, both text messages and media files.
- Simulated human system users, both as desktop users and system administrators.

A Python script produces the simulated messages. They are sent over a message queue to the C2 application. They are sent every 10-20 seconds and contain information on the sender and the vehicle's current position. The position is sent along a simulated route, with the starting position set to the simulation's start. Media files are sent to an FTP server on the media client every 60-120 minutes. The media file to send is selected randomly from a library of files.

The simulated users differ between the different variants of the attack executed in the simulation. There are two types of simulated users: desktop users and system administrators. Desktop users have the following features:

- They use the C2 system's web interface (monitor) to read messages from vehicles.
- They use the C2 system's web interface (monitor) to send commands to vehicles.
- They open files on the local machine and on the media server.
- They read and send emails to each other using an email client.

The system administrators execute code on machines using *Psexec* (i.e., remote execution of commands) and connect to servers via remote desktops.

All action sequences, including their pauses, are predefined based on schedules containing actions, their parameters, and an execution time relative to the start of the simulation. These determine what the simulated users should do at different points in time. The probabilities of each action and the sleep time are randomised from intervals set to correspond to a typical workday of an officer or administrator. In nine simulations, a Markov model is used to produce the schedules, and the previous task influences the probability of the next task. In the tenth variant, a simple Gaussian model is used to select tasks randomly from a probability distribution. In six of the schedules, the normal work tempo is occasionally changed to a more intense one, in which the simulated workers work nonstop. This is to simulate an important event in the operation. In three of the schedules, the probabilities shift over time so that desktop users tend to use the C2 application more, and system administrators tend to work less intensively as time progresses. These shifts in behaviour are intended to make room for training detection models that can handle concept drift (an objective of one of the AIInception detectors) and can be seen as a reflection of a more intensive operation. They reflect the increased importance of the operation and the system administrators' satisfaction with the setup of the cyber infrastructure at the temporary headquarters.

Because of the different schedules and randomisation procedure, the intensity at which users work is varied. The typical schedule involved approximately 250 actions per day, executed over 10 hours with a lunch break in the middle of the work shift. Logs from interactions with simulated users are included in the dataset. An excerpt of a log entry is presented below:

```
{ "data": "{ \"event\": \"agent_message\", \"data\": \"{ \"ID\": 391, \"State\": 9,
\"Message\": \"Have read 1 emails.\" } }\", \"id\": 5874718, \"log_source_id\": -1,
\"log_source_type\": \"CrateBot\", \"status\": \"EntityState.RUNNING\",
\"time_stamp\": \"2024-10-31 08:27:22\" },

{ \"data\": \"{ \"event\": \"Command completed\", \"command\": { \"ActionType\":
\"ReadMail\", \"RelativeExecutionTimeFromStart\": 150857, \"FileFolder\":
\"media.bnhq2.rotsoreports\", \"EmailsToRead\": 1, \"EmailProcessing\":
\"ClickLinks\", \"ID\": 391}, \"log_count_command\": 4, \"completed_commands\": 392,
\"log_count_total\": 915974, \"command_run_time\": 30.538971185684204 }\", \"id\":
5874719, \"log_source_id\": -1, \"log_source_type\": \"CrateBot\", \"status\":
\"EntityState.RUNNING\", \"time_stamp\": \"2024-10-31 08:27:22\" },
```

Malicious events in the cyber infrastructure

The attack in SL300 includes three sequences:

- **Initial access** by the implant spreading from a vehicle to the media server using the MITRE ATT&CK technique *T1021 (Remote Services)* and a shared password. From the media server, it establishes a command-and-control channel to external networks.
- **Lateral movement** from the media server to the C2 application using a vulnerability in the message queuing software. It establishes another command-and-control channel and begins investigating the application server (e.g., searching for files and exfiltrating findings).
- **Impact** by destroying the functionality of the C2 application. This is done with a wiper that corrupts the settings of the machine and makes it unusable.

The malicious events were simulated using the tool *Lore*. Each simulation has several variations in how the attack is executed. This variation consists of different methods for initial access (PsExec or WMI); differences in how networks are scanned; differences in which part of the C2 application was targeted (monitor or apps); and differences in how long it waits between activities. The variants took between 30 and 240 minutes to reach the final state, at which point the application is unusable.

Logs from each execution are included in the produced dataset. They consist of time-stamped actions. An example of Lore running *nmap* for network discovery is included below. In this example, the scan took approximately 38 minutes to complete and was executed using system credentials on the *apps* machine. As the log shows, this remote shell is of type *meterpreter* with a connection to 25.18.17.80:4447 via 52.41.202.11:50675, and had been active for more than approximately 45 before the scan took place:

```
{
  "action_uid": "NmapFromCompromisedMachine",
  "action_category": "Metasploit.Shellcode",
  "action_description": "Nmap run from a compromised machine. If the target
                        does not have nmap installed, an appropriate version
                        of nmap is transferred to the target and executed
                        through a live metasploit session to the machine.",
  "action_outcome": "ERROR",
  "action_info": {},
  "mitre_att&ck": {
    "techniques": [
      "T1046"
    ],
    "tactics": [
      "TA0007"
    ]
  },
  "time_start": "2024-11-05 11:00:44",
  "time_end": "2024-11-05 11:38:36",
  "c2_info": {
    "session_host": "192.168.4.20",
    "shell_user": "nt authority\\system @ apps",
    "shell_type": "meterpreter",
    "tunnel_local": "25.18.17.80:4447",
    "tunnel_peer": "52.41.202.11:50675",
    "active_since": "2024-11-05 10:17:53",
    "established_via": "exploit/multi/misc/apache_activemq_rce_cve_2023_46604"
  }
}
```

Approximately 200 such entries are in each attack log. It should be noted that the attacks were allowed to continue after the impact was reached. In addition, no consideration was

taken to how the attacks were to circumvent responses that could be taken, because no responses were simulated.

In one of the attack simulations, an unexpected event sequence unfolded. The attacker obtained control of the domain controller on the network using hashed credentials. This was not planned for. Our analysis suggests that these were obtained through *Lore*'s interaction with simulated users, specifically by listening to and recording remote desktop sessions or *PsExec* commands executed on the media server. This was not part of the simulation plan. However, it did not disrupt the planned sequence of events and is not deemed problematic.

System and network logs

All machines in the BNHQ were configured to send host logs. For Windows machines, these logs included Windows event logs, as determined by the Microsoft "Stronger Recommendation" configuration, and Sysmon logs, as determined by the popular SwiftOnSecurity configuration. For Linux machines, *auditd* was running the same configuration as used as for SL700. All logs stored in *"/var/logs"* were collected from Linux machines. Network events were recorded using *Suricata* using a ruleset from *Emerging Threats*. In addition, *Suricata* was configured to record NetFlow. Due to traffic volumes, e.g. caused by media files, full packet captures were not included.

The approximate number of entries from the log sources in the dataset is shown in table 2. Note that these are rough estimates. Training data here refers to the five days with no attacks; test data refers to the part of the day with an attack.

Table 2. Overview of logs collected in the simulations.

Log source	Approximate number of entries in training data	Approximate number of entries in test data
Windows events, incl Sysmon	60 million	10 million
Linux host logs events	10 million	2 million
Suricata events, mostly netflow	250 million	50 million

All logs were extracted and stored in text files, one log entry per row, written as JSON in the *Elastic Common Schema* (ECS) format. Events known to be direct artefacts of the simulated users' instrumentation were removed. An example of a Windows log from the office machine *hq08* is included below:

```
{
  "ProcessId": "0x794",
  "NewProcessId": "0x1854",
  "TargetDomainName": "-",
  "SubjectUserName": "urbgro",
  "TargetLogonId": "0x0",
  "TargetUserSid": "S-1-0-0",
  "SubjectUserSid": "S-1-5-21-2140222656-1986183540-1830489456-1123",
  "MandatoryLabel": "S-1-16-8192",
  "SubjectDomainName": "office",
  "TokenElevationType": "%1936",
  "NewProcessName": "C:\\Program Files (x86)\\Mozilla Firefox\\pingsender.exe",
  "ParentProcessName": "C:\\Program Files (x86)\\Mozilla Firefox\\firefox.exe",
  "TargetUserName": "-",
  "SubjectLogonId": "0xe8605",
  "process": {
    "thread": {
      "id": 4552,
      "pid": 4
    }
  },
  "host": {
    "name": "hq08.office.bnhq1.rotso",
    "agent": {
      "ephemeral_id": "0f81151d-819c-4daf-bb59-0c813aefclbf",
      "id": "3759a057-fc6b-432d-b8c0-45cabae0ca4f",
      "version": "8.10.3",
      "name": "hq08",
      "type": "winlogbeat"
    },
    "log": {
      "level": "information",
      "event": {
        "outcome": "success",
        "created": "2024-10-30T05:30:49.393Z",
        "code": "4688",
        "provider": "Microsoft-Windows-
```

```
Security-Auditing", "kind": "event", "action": "Process Creation"},
"@timestamp": "2024-10-30T05:30:47.776Z", "ecs": {"version": "8.0.0"},
"message": "A new process has been created.\n\nCreator Subject:\n\tSecurity
ID:\t\tS-1-5-21-2140222656-1986183540-1830489456-1123\n\tAccount
Name:\t\tturbgro\n\tAccount Domain:\t\ttooffice\n\tLogon
ID:\t\t0xE8605\n\nTarget Subject:\n\tSecurity ID:\t\tS-1-0-0\n\tAccount
Name:\t\t-\n\tAccount Domain:\t\t-\n\tLogon ID:\t\t0x0\n\nProcess
Information:\n\tNew Process ID:\t\t0x1854\n\tNew Process Name:\tC:\Program
Files (x86)\Mozilla Firefox\pingsender.exe\n\tToken Elevation
Type:\t\t%1936\n\tMandatory Label:\t\tS-1-16-8192\n\tCreator Process
ID:\t\t0x794\n\tCreator Process Name:\tC:\Program Files (x86)\Mozilla
Firefox\firefox.exe\n\tProcess Command Line:\t\n\nToken Elevation Type
indicates the type of token that was assigned to the new process in
accordance with User Account Control policy.\n\nType 1 is a full token with
no privileges removed or groups disabled. A full token is only used if User
Account Control is disabled or if the user is the built-in Administrator
account or a service account.\n\nType 2 is an elevated token with no
privileges removed or groups disabled. An elevated token is used when User
Account Control is enabled and the user chooses to start the program using
Run as administrator. An elevated token is also used when an application is
configured to always require administrative privilege or to always require
maximum privilege, and the user is a member of the Administrators
group.\n\nType 3 is a limited token with administrative privileges removed
and administrative groups disabled. The limited token is used when User
Account Control is enabled, the application does not require administrative
privilege, and the user does not choose to start the program using Run as
administrator."}
```

SL700

This storyline models a simulated military garrison and its digital infrastructure, including IT and OT components such as firewalls, DNS servers, and surveillance systems. The simulation reproduces normal garrison operations as well as a coordinated cyber–physical intrusion in which an adversarial actor gains access through network exploitation and insider assistance.

The virtual environment enables detailed observation of how the garrison’s systems respond to compromise, including effects on surveillance, authentication, and operational control networks. It provides a comprehensive setting for analysing cyber–physical interactions, situational awareness, and system resilience under realistic operational conditions.

The simulated attack chain demonstrates how an adversary gains control over a critical network component to intercept a video stream. It begins with network scanning and exploitation of a server vulnerability to establish remote access, followed by privilege escalation using multiple variant techniques. The attacker then installs persistence mechanisms, evades detection, and ultimately manipulates firewall rules to disrupt or block the video feed. The accompanying network diagram visualises the emulated infrastructure and its interconnections.

SL700 topology infrastructure was captured in a graph database using Neo4j, in the same way as for SL100 (*SL700_infra.json* in the SL700 ZIP files). Each infrastructure node includes fields such as name, infrastructure_types and id that describe its role in the topology. Examples include the central firewall *FW-SL700*, multiple routing segments such as *VPN-NET*, *DMZ*, and *LAN*, and devices deployed within those segments, including *VPN-ROUTER*, various workstation-type hosts (e.g., *video_server*, *cas2*, *admin_pc2*, *kafka*), and specialized devices like *cam*. The corresponding IP nodes describe networks such as 192.168.10.0/24, 172.17.100.0/24, and 192.168.100.0/24, as well as the specific addresses used by each device.

Relationships define how these objects fit together. *consists_of* links associate devices with their IP addresses and the network blocks they belong to, making the subnet structure explicit. *communicates_with* relationships describe connectivity between routers and their associated subnets, between devices and the routing infrastructure, and between components within the same logical segment. Together, the nodes and relationships form a complete graph-based model of the SL700 network layout, enabling analysts to understand communication paths, device placement and subnet boundaries that support the storyline and simulated activity within this environment. Figure 3 shows the network topology of SL700.

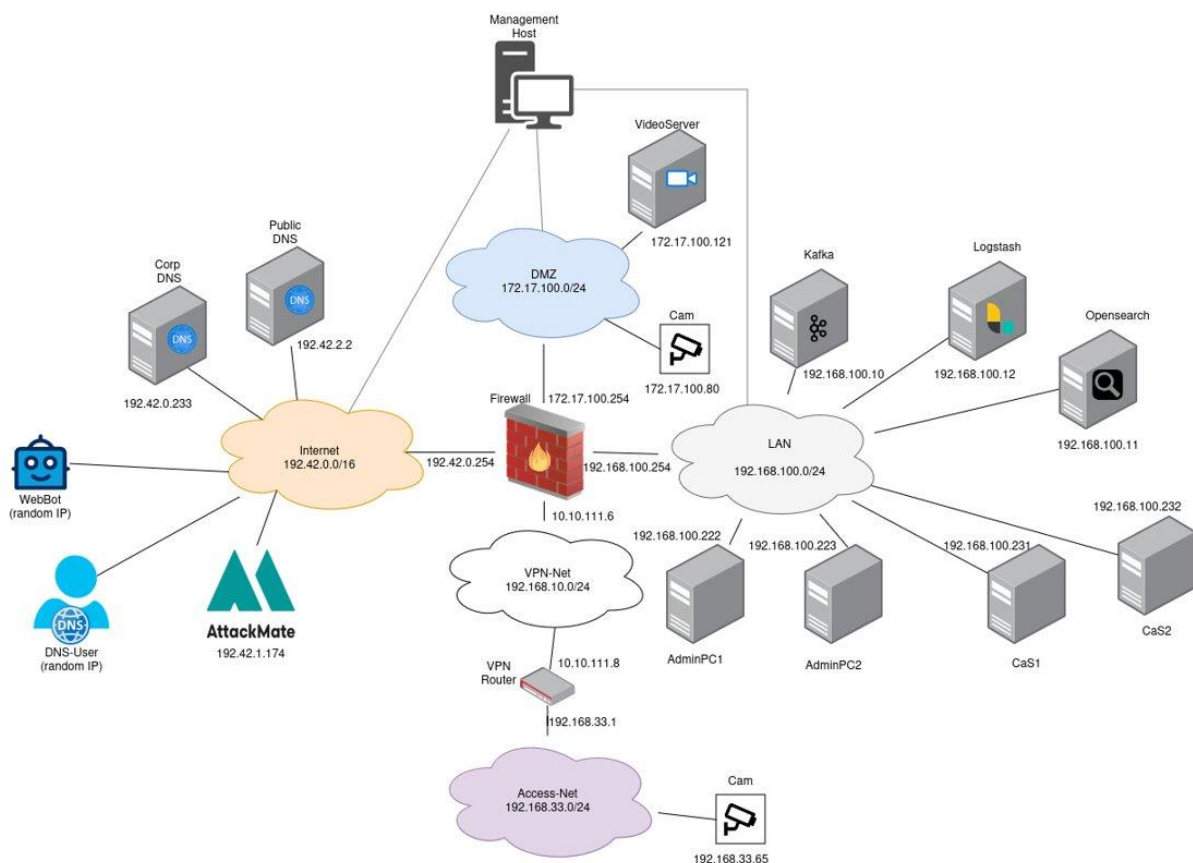


Figure 3. Network topology SL700.

Structure of the ZIP files

The dataset consists of 15 ZIP files, each containing a variant of the simulation:

- There is one variant of SL100.
- There are eight variants of SL300.
- There are six variants of SL700.

The structure and content will vary somewhat between the storylines and variants. Common to all ZIP files are the following types of logs:

- **Logs:** Raw logs captured from monitoring e.g. network traffic and hosts.
- **Labelled logs:** Labelled logs (e.g. as malicious or benign, possibly labelled with underlying MITRE ATT&CK technique)

- **Alerts:** Alerts generated by applying an intrusion detection system (IDS) to the logs (possibly an IDS developed in the AIInception project)

Separate sections below detail the particularities of each storyline. Note that all ZIP files contain a *readme.ML*

SL100

SL100 contains mainly system logs and Netflows gathered from the SL100 topology. More specifically, the (uncompressed) SL100 ZIP file has the following structure:

```

├── SL100/
│   ├── README.md
│   ├── sl100-labeled-data
│   │   ├── audit.log
│   │   ├── drone_flight_logs.log
│   │   ├── fast.log
│   │   ├── filtered_nteflow_data_labeled.json
│   │   ├── sl100_1.json
│   │   ├── WIN2.txt
│   │   └── WSUS.txt
│   ├── sl100-raw-data
│   │   ├── audit.log
│   │   ├── drone_flight_logs.log
│   │   ├── fast.log
│   │   ├── filtered_nteflow_data_labeled.json
│   │   ├── sl100_1.json
│   │   ├── WIN2.txt
│   │   └── WSUS.txt
│   ├── alerts
│   │   ├── sl100_alerts.json
│   │   ├── sl100_alerts
│   │   │   └── sl100_alerts.json
│   └── SL100_infra.json

```

The *sl100_1.json* file (see below) contains all the information on the actions, both benign and malicious, performed to generate the dataset. The original raw data are all included in the *sl100-raw-data* folder. The events in each file were then structured and labelled as benign or malicious, based on the description in *sl100_1.json*. The labelled events can be found in the *sl100-labeled-data* folder. The *sl100-raw-data* and *sl100-labeled-data* folders contain:

- *drone_flight_logs.log*: The flight logs of the drone with information on height, speed, location etc.
- *audit.log*: Security logs from the ubuntu machine hosting the drone simulator.
- *WSUS.txt*: Sysmon logs from the WSUS machine.
- *WIN2.txt*: Sysmon logs from the WIN2 machine.
- *fast.log*: Suricata logs containing intrusion detection alerts.
- *filtered_netflow_data.json*: Suricata netflows extracted from the eve.json file
- *sl100_1.json*: The file discussed above containing all the steps we performed to gather the logs.

The *alerts* folder includes the alerts produced by a detector called DroneShield that has been developed in AIInception. *DroneShield* classifies events from SL100 as benign or malicious. *sl100_alerts.json* includes 38076 alerts and each alert has the following format:

- “*UniqueIDGeneratedByTheDetector*”: A unique alphanumeric string,
- “*detector*”: "DroneShield",
- “*malicious*”: 0 for benign and 1 for malicious event,
- “*log_file*”: The file (audit.log, drone_flight_logs.log, fast.log etc.) from which an event was processed,
- “*timestamp*”: Time of the event,
- “*confidence*”: A confidence score, indicating the certainty of the tool's decision}

Finally, *SL100_infra.json* provides a graph of the SL100 topology that was used to create the dataset. This has been described above.

SL300

As already discussed, variant1 and variant3 are not included because their network logs were not captured properly. With the exception of variant2 and variant5, each of the variants have the following structure:

```
variantX/
├── alerts/
├── cyber_terrain/
├── attack_log/
│   └── sl300_X.json
├── labelled_logs/
│   ├── bnhqX/
│   │   ├── attack/
│   │   └── benign/
├── logs/
│   ├── attack_day/
│   └── benign_days/
├── MAL/
│   ├── mal/
│   ├── MAL_view_02_17_09
│   └── AttackGraph-SL300.json
├── simulated_user_log/
│   ├── sl300_5_attack_day_markov_daily_analytics.pkl
│   └── sl300_5_normal_days_markov_daily_analytics.pkl
└── readme.md
```

Each variant has a folder with the *alerts*. Each of them has a different set since some detectors could not be used for certain runs of the storyline. The *attack_log* folder contains a JSON export from the tool *Lore*, detailing the attacks executed in the storyline. The format is

intended to be self-explanatory. All attack logs correspond to events that occur during the *attack day*. The *simulated_user_log* folder contains logs exported from the SVED tool that describe the actions performed by simulated users. One file covers the *benign days* and another the *attack day*. The *cyber_terrain* folder contains information about the network used in all simulations. Note that the data was collected for variant1, but all networks are identical except for their names (i.e., the numbers in the FQDNs).

MAL

In order to create an attack graph model, we utilized the Meta Attack Language framework [4], which was initially introduced as a meta-domain specific language (DSL) for modelling infrastructures with domain-specific threats. MAL combines elements of UML classes and object diagrams with probabilistic attack-defence graphs. The language uses *Assets*, which can be thought of as a grouping of classes that contain instances and represent objects. *Assets* can also be interpreted as set of nodes in an attack graph. Those sub-graphs (*Assets*) are connected to other sub-graphs via *Associations*. *Associations* define relationships similar to those in class diagrams, including feature roles and multiplicities. *Assets* and *Associations* are defined in MAL files, which must follow a specific syntax. To compile a MAL file, the MAL compiler [5] must be used. In our case, we use the compiler without any modifications.

The MAL compiler compiles MAL specifications (.mal files) into different formats, using different backends. The reference backend generates Java code that is suitable for testing purposes and evaluating your language.

To use this, execute the following steps to compile a MAL Instance with the MAL compiler. The MAL instance, in this case, is the *exampleLang* given by the official MAL Repository [8].

```
git clone git://github.com/mal-lang/malcompiler.git
cd malcompiler
mvn install
```

To compile *exampleLang* into JAVA files, execute the following command:

```
mvn generate-test-sources
```

To compile *exampleLang* into JAVA files and then compile these JAVA files and the unit tests in *src/test/java* into CLASS files, execute the following command:

```
mvn test-compile
```

To run a specific test class, execute the following command:

```
mvn test -Dtest=TestExampleLang
```

Here, *TestExampleLang* is the test class. To run a specific test method in a test class, execute the following command:

```
mvn test -Dtest=TestExampleLang#testNoPassword
```

Here, *TestExampleLang* is the test class and *testNoPassword* is the test method. More details can be found in [9]. At this point, one can proceed with creating the attack graph. The process is detailed in tutorial provided by the official GitHub repository [10].

AttackGraph_SL300.json is the final output from the AIInception MAL Attack graph generator, which was developed using the *MAL Toolbox* Python library [6]. To visualise and show the connections in the MAL language specification, we used the MAL visualisation tool [7], which generated the *MAL_view_02_17_09* folder. The folder contains an *index.html* file that

visualises MAL assets and the Associations. Note that the MAL directory is identical for each variant.

Variant 2 (SL300)

The only difference with variant 2 is one additional file as highlighted below:

```
├── variant2/
│   ├── alerts/
│   ├── cyber_terrain/
│   ├── attack_log/
│   │   └── sl300_2.json
│   ├── labelled_logs/
│   │   ├── bnhq2/
│   │   │   ├── attack/
│   │   │   └── benign/
│   ├── logs/
│   │   ├── attack_day/
│   │   └── benign_days/
│   ├── MAL/
│   │   ├── mal/
│   │   ├── MAL_view_02_17_09
│   │   └── AttackGraph-SL300.json
│   ├── simulated_user_log/
│   │   ├── sl300_5_attack_day_markov_daily_analytics.pkl
│   │   └── sl300_5_normal_days_markov_daily_analytics.pkl
│   ├── alert_groups_delta_3_seconds.pkl
│   └── readme.md
```

This variant (and variant 5) has been used more extensively than the other variants. As a result, they have more comprehensive detector coverage and more alerts.

The additional PKL file contains the *alert groups*, which has been created from the alerts over several iterations. Here, the alert sets from the AMiner, Telosian and CEA detectors have been stitched together to form graph-based alert groups, whose graph structure is preserved in the group.² The alert identifiers found in these groups refer to the same identifiers found in the alert sets themselves (although they are found under different JSON keys in different sets). The cutoff time for these alert groups was three seconds, but for optimal detection results, we used different cutoff times for different attack classes. These groups serve as an illustrative example. For details of the graph-based alert group method we refer to [3].

Variant 5 (SL300)

Variant 5 of SL300 has been used most extensively during the project, and as a result, is the most complete variant. It has the following structure, with new elements highlighted:

² For this we have been using Python's networkx graph format.

```

|— variant5/
|   |— alerts/
|   |— cyber_terrain/
|   |— attack_log/
|   |   |— sl300_5.json
|   |— labelled_logs/
|   |   |— bnhq5/
|   |   |   |— attack/
|   |   |   |— benign/
|   |— logs/
|   |   |— attack_day/
|   |   |— benign_days/
|   |— MAL/
|   |   |— mal/
|   |   |— MAL_view_02_17_09
|   |   |— AttackGraph-SL300.json
|   |— simulated_user_log/
|   |   |— sl300_5_attack_day_markov_daily_analytics.pkl
|   |   |— sl300_5_normal_days_markov_daily_analytics.pkl
|   |— alert_groups_delta_3_seconds.pkl
|   |— ioc.json
|   |— stix.json
|   |— SL300_threat_report.pdf
|   |— SL300_output_1BS.json
|   |— sl300_kg.dump
|   |— readme.md

```

As for variant 2, it contains the alert groups as a PKL file.

The *ioc.json* file contains IOCs such as network-related data (IP addresses, protocols, etc.) extracted from alerts generated by Aminer and Telosian, along with enrichments and correlations with past events using MISP platforms.

The *stix.json* file contains indicators of compromise extracted from the Aminer and Telosian, represented as STIX objects along with their relationships and bundles.

SL300 includes a threat report generated by an (fictitious) intelligence agency. This is included in the AlInception scenario and storyline report [1] and slightly adapted here. *SL300_threat_report.pdf* contains this report. From this report, we have extracted structured cyber threat intelligence (CTI) as STIX objects using the tool and technique described in [11]. This is represented in JSON in the file *SL300_output_1BS.json*. This file is the outcome of the process taking the PDF as input, splits it by pages and extracting the raw text. It runs CTI-oriented NER on the text to detect entities such as threat actors, malware, campaigns, attack patterns, file paths / IoCs, etc. For each detected entity it also infers relations (“uses”, “authored-by”, “attributed-to”, “indicates”, “drops”, “communicates-with”...), building small *source-relation-target* triplets. It enriches each text chunk with these entities and relations in a structured JSON format, keeping references to file name, page and character offsets.

Finally, both SL100 and SL700 contained a (minimalistic) knowledge graph with the topology. For this variant we have developed a more complete Cyber Security Knowledge Graph (CSKG) that integrates

data from standards like MITRE ATT&CK, CAPEC, SL300's topology elements (IP addresses and devices) and outputs of tools used in the AInception framework, like alerts from intrusion systems, alert groups from grouping tool, and indicators of compromise from the CTI enrichment tool described above. Core entities and relationships:

1. **Alert** (security event or detection generated by an IDS)
Key properties:
Timestamps (detection timestamp), alert_type (name of the IDS that produced the alert), src_ip (alert's source IP), dest_ip (alert's destination IP)
Key relationships:
Alert – SOURCE_IP --> IPv4Address
Alert – DEST_IP --> IPv4Address
Alert <-- CONTAINS – AlertsGroup
2. **AlertsGroup** (entity resulted from alert grouping tool)
Key property:
alert_list (list of IDs of the alerts in the group), size (how many alerts are in this group)
Key relationship:
AlertsGroup -- CONTAINS --> Alert
3. **IPv4Address**
Key property:
value (IP address or CIDR)
Key relationships:
IPv4Address <-- consists_of -- Infrastructure (workstation or firewall within the infrastructure)
IPv4Address <-- consists_of -- IPv4Address (CIDR node)
IPv4Address (CIDR node) <-- communicates_with – Infrastructure (Router/Switch within the infrastructure)
4. **Infrastructure**
Key property:
infrastructure_types (firewall, workstation or routers-switches)
Key relationships:
(see IPv4Address relationships)
5. **X_CVE** (vulnerabilites)
Key properties:
Description, name (e.g. CVE-2014-4696)
Key relationships:
X_CVE – uses --> X_CWE (weaknesses)
X_CVE – affects--> X_CPE (software or hardware)
6. **X_CWE** (Common Weakness Enumeration)
Key properties:
x_name (name of the weakness)
x_extended_description (text describing the CWE)
Key relationships:
X_CWE – is_child_of --> X_CWE
X_CWE – RELATED_ATTACK_PATTERN --> AttackPattern

X_CWE – HAS_EXTERNAL_REFERENCE --> ExternalReference (holds the CWE code name, e.g. CWE-165)

7. **X_CPE** (Common Platform Enumeration)

Key properties:

cpe (the official CPE string, e.g. cpe:2.3:a:mozilla:firefox:68.2.0:*:*:*:*:*)

Key relationship:

(see X_CVE Key Relationships)

8. **Malware**

Key properties:

Malware – uses --> AttackPattern

Malware – indicates --> ThreatActor

Malware – exploits --> X_CVE

Malware – revoked-by | controls --> Malware

Malware – MENTIONED_IN --> Alert

This is stored in the *sl300_kg.dump* file.

SL700

This dataset contains simulated benign and malicious system/network/OT logs for the SL700 storyline (garrison/video-server attack chain). The simulation follows the SL700 storyline and AttackMate-driven attack scripts. The dataset includes raw recordings per host (logs + configs + facts), AttackMate execution logs (human-readable + JSON), and a separate labelled subset (JSON event objects for selected sources).

Log sources

The dataset collects the typical host, application and network sources produced during the simulation runs. Example sources collected per host include:

Host logs

Apache access and error logs.

- Audit logs.
- Authentication logs, syslog, messages, kern.log.
- Zoneminder application logs.
- Collectd logs, etc.

Network logs

- PCAPs and netflow exports.
- Suricata alerts.
- VPN logs, firewall logs.

Attack instrumentation

- attacker/logs/attackmate.log (human-readable AttackMate run).
- attacker/logs/attackmate.json (structured events with MITRE technique labels).

Storyline & attack steps

1. Reconnaissance

- DNS enumeration, port scans (nmap), web vulnerability scans (Nikto), content fuzzing (ffuf).

2. Initial access

- Exploit of ZoneMinder (vulnerable versions) to get a reverse shell + foothold (T1190). Facts file documents Zoneminder version for correlation.

3. Post-exploit and tool transfer

- Upload/execution of tools (e.g., linpeas) (T1105) and meterpreter payloads; AttackMate records payload & options.

4. Privilege escalation (variants implemented)

- a) PwnKit (Polkit / pwnkit)
- b) sudo misconfiguration/weakness
- e) misconfigured cron-job
- f) found SSH key (local credential)

5. Backdoor persistence

- a) Add SSH key to authorized_keys
- b) Create a new account on the machine
- c) Modify PAM (pam-backdoor style)

6. Evasion & impact

- Command splitting, scheduled triggers, and finally iptables modification to block CCTV streams (Denial of View).

Runs executed include:

- (a, a) — normal+attack (2 h normal + 2 h attack).
- (a, b) — normal+attack (2 h normal + 2 h attack).
- (a, c) — normal+attack (4 h normal + 2 h attack).
- (b, a) — normal+attack (2 h normal + 2 h attack).
- (e, a) — normal+attack (2 h normal + 2 h attack) — includes Netflow collection.
- (f, a) — normal+attack (2 h normal + 2 h attack).

Labeled data

- **Location:** same host folder structure but contains only selected log sources (auth, syslog, audit, pcaps) and no configs or facts.
- **Format:** Each labelled entry is a JSON object per event, for example:

```
{
  "@timestamp": "2024-11-13T14:53:25.518000Z",
  "event": {"type": "EXECVE"},
  "message": "argc=14 a0=\"sudo\" ...",
  "malicious": 1,
  "attack_description": "A ssh command was executed.",
  "mitre_techniques": ["T1120"],
  "mitre_tactics": ["Discovery", "Defense Evasion"]
}
```

- **Important:** The labeling process in the provided labeled folder is not fully reliable. Use `gather/attacker/logs/attackmate.json` for accurate label derivation.

File structure

There are six simulation runs provided in the data set. They are named `run_<v1>_<v2>`, where `<v1>` and `<v2>` are identifiers for variants of the simulation, such as `run_a_a`. For more information on the variations we refer to the attack description above.

Each run contains the raw logs captured during the simulation `attack_<v1>_<v2>`, labelled log data `attack_<v1>_<v2>_labeled`, and alerts collected from anomaly-based intrusion detection systems (directory `/alerts`). It has the following structure:

```

|— run_<variant1>_<variant2>/
|   |— alerts/
|   |   |— aminer_alerts.log
|   |   |— telosian_alerts_clean.json
|   |
|   |— attack_<variant1>_<variant2> /
|   |   |— gather/
|   |   |   |— adminpc1/
|   |   |   |   |— configs/
|   |   |   |   |— logs/
|   |   |   |   |— facts.json
|   |   |   |— adminpc21/
|   |   |   |— attacker/
|   |   |   |   |— configs/
|   |   |   |   |— logs/
|   |   |   |   |— facts.json
|   |   |   |— caspc1/
|   |   |   |— caspc2/
|   |   |   |— cordpns/
|   |   |   |— dnsuser1/
|   |   |   |— ghosts/
|   |   |   |— inetfw/
|   |   |   |— kafka/
|   |   |   |— logstash/
|   |   |   |— opensearch/
|   |   |   |— videoserver/
|   |   |   |— webcam/
|   |   |   |— webuser1/
|   |   |   |— etc_hosts
|   |   |   |— gather.yml
|   |   |   |— hosts
|   |   |   |— requirements.yml
|   |   |— attack_<variant1>_<variant2>_labeled/
|   |   |   |— attack_<variant1>_<variant2> /
|   |   |   |   |— gather/
|   |   |   |   |   |— adminpc1/
|   |   |   |   |   |   |— logs/log/
|   |   |   |   |   |   |   |— auth.log.1.json
|   |   |   |   |   |   |   |— auth.log.2.gz.json

```

```

├── syslog.1.json
├── syslog.2.gz.json
├── adminpc21/
├── attacker/
├── caspc1/
├── caspc2/
├── cordpns/
├── dnsuser1/
├── ghosts/
├── inetfw/
├── kafka/
├── logstash/
├── opensearch/
├── videosever/
├── webcam/
├── webuser1/
├── readme.md
├── SL700 infra.json

```

Folders in the `attack_<variant1>_<variant2>/gather/` directory follow the same structure as `adminpc1/`. Similarly, folders in the `attack_<variant1>_<variant2>/labeled/` directory follow the same structure as `adminpc1/`.

Important files and directories

- `gather/<hostname>/facts.json` — per-host facts (installed packages, versions, IPs, OS info).
- `gather/attacker/logs/attackmate.json` — canonical attack timeline and metadata (start-datetime, cmd, techniques, tactics, payload/options).

Alerts

For each variant, the alert sets from the detectors used in that run are included. For SL700, this was limited to two detectors: AMiner and Telosian. Suricata alerts are also present in the log set for these runs and were used alongside the other alerts for alert grouping, but have not been included in the alert files

Summary

The AIInception dataset comprises a total of 15 different simulations which are variants of three military storylines developed in the AIInception project. Whilst the dataset has several limitations, it is unique in that it bridges a high-level geo-political scenario – through the operational and tactical levels, as detailed in the timeline and storylines – to their technical simulation and application as detailed in this dataset.

References

- [1] A. Thuv (editor) et al. Alnception scenario and storylines, FFI-report 25/039, Available from <https://www.ffi.no/en/publications-archive/alnception-scenario-and-storylines>, 2025
- [2] <https://ardupilot.org/>
- [3] Magnus Wiik Eckhoff, Peter Marius Flydal, Siem Peters, Martin Eian, Jonas Halvorsen, Vasileios Mavroeidis and Gudmund Grov, *A Graph-Based Approach to Alert Contextualisation in Security Operations Centres*, in Proc. 28th Information Security Conference (ISC 2025), https://doi.org/10.1007/978-3-032-08124-7_24
- [4] <https://github.com/orgs/mal-lang/repositories>
- [5] <https://github.com/mal-lang/malcompiler>
- [6] <https://github.com/mal-lang/mal-toolbox>
- [7] <https://github.com/mal-lang/MAL-visualization>
- [8] <https://github.com/mal-lang/exampleLang>
- [9] <https://github.com/mal-lang/mal-documentation/wiki/Compiling>
- [10] <https://github.com/mal-lang/mal-toolbox-tutorial>
- [11] O.Sorokoletova, E. Antonioni and G. Colò. *Towards a scalable AI-driven framework for data-independent Cyber Threat Intelligence Information Extraction*, 2nd International Conference on Foundation and Large Language Models (FLLM), IEEE, 2024.